The face my AI never looked at

My app scans 5 frames a second to stay fast. On a busy street, a man's face came out between two heads for a sixth of a second, right between two scans: no box, no blur. Here's how a person catches it, what stays blurred, and how the export got fast.



C# .NET 10 WPF OpenCV FFmpeg H.264

Samit Desai · linkedin.com/in/samit-desai-62868420

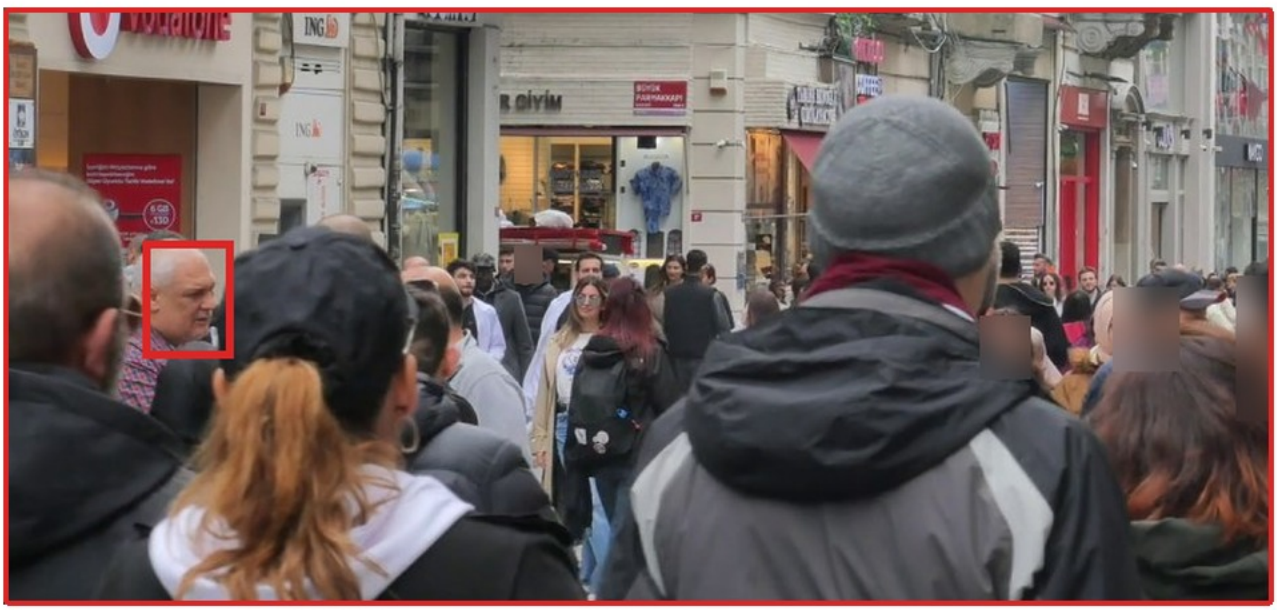
How often the AI looks: you choose

1. **Every frame**
   slowest, full coverage: for footage where nothing may slip through
2. **1 of every 2 frames**
   half the scans
3. **10 frames a second**
   a busy street
4. **5 frames a second**
   the default, and it leaves gaps
5. **2 frames a second**
   fastest: little moves, like a podcast or a meeting room

It's a trade-off between time and safety.

A face between two scans

On the same street as Episode 3, the scan looks at Frame 217, then at Frame 223. In between, this man's face comes back between two heads for five frames.



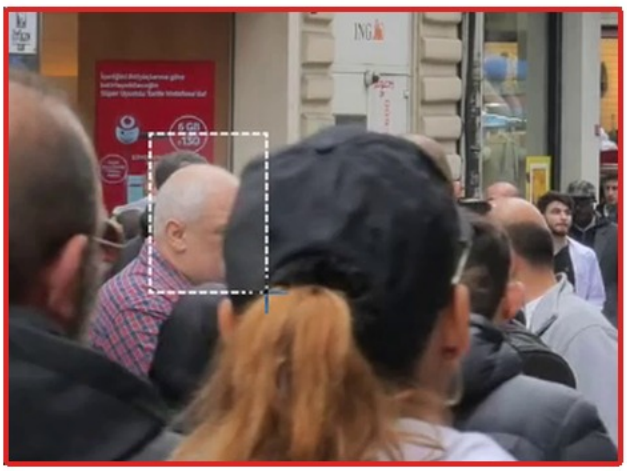
Frame 220: his face, outlined in red. No box, nothing blurs it.



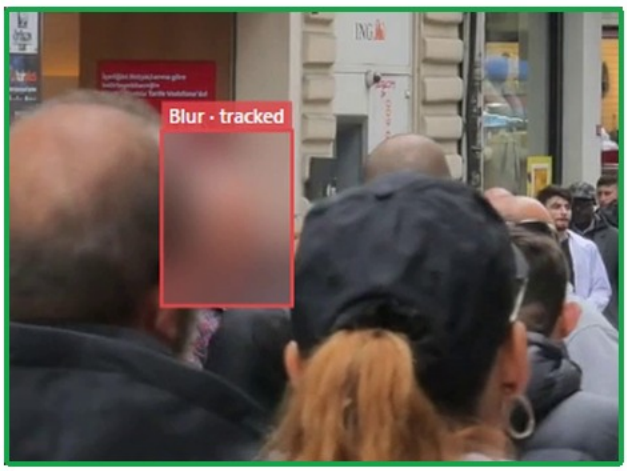
217	218	219	220	221	222	223
scan	face	face	face	face	face	scan

- Five frames, a sixth of a second. No scan lands on them, so there's no box, and nothing blurs him, not even in the export.
- A viewer asked about exactly this: can a reviewer catch it without scanning the whole video again?

Yes: a person fills the gap



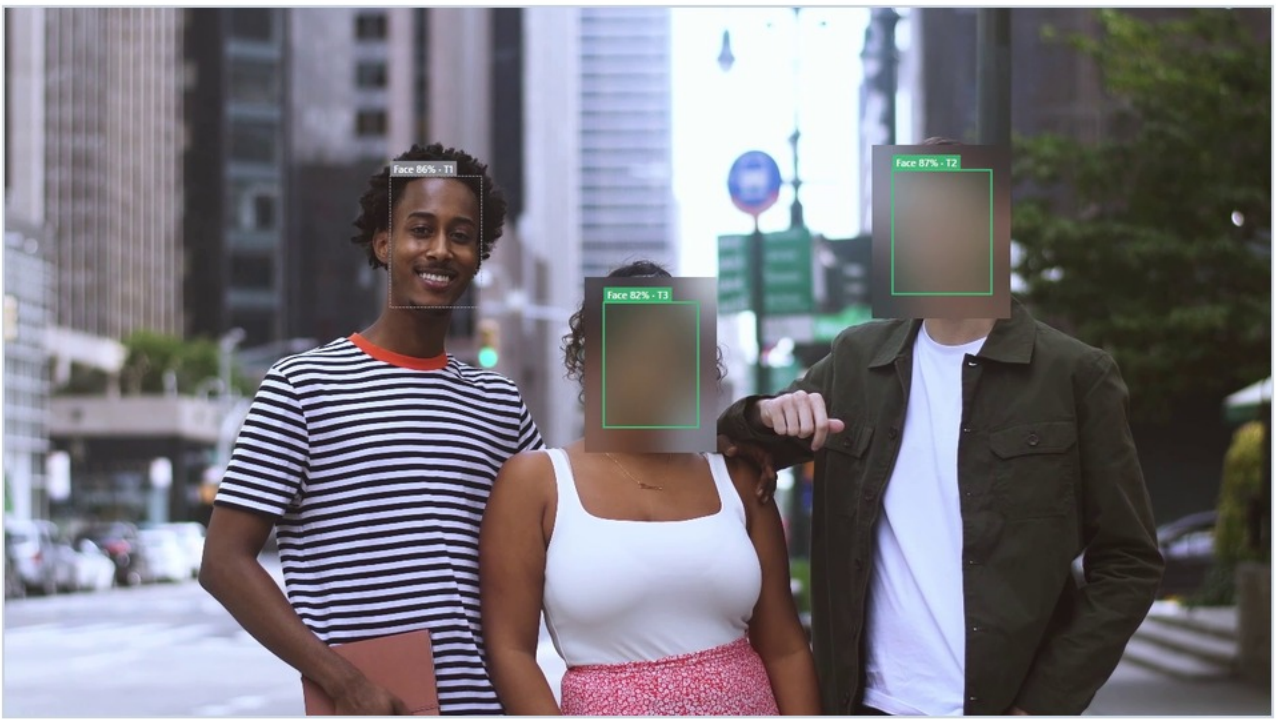
Frame 218: draw a box



Frame 222: still tracked

- Step frame by frame with the arrow keys. When his face appears, draw a box around his head and press Track.
- OpenCV's KCF tracker follows it forward and loses him when he goes behind the next head, so the region ends right there: he's blurred on every frame where he shows.
- Or press D: the model runs on just this one frame.
- Or scan again at Every frame: slower, but then the AI looks at his face too.

What stays blurred: two cases



- A face the AI found starts as pending, and pending is blurred. Approving only records that someone checked it; only a reject shows a face, like the one here.
- So even an unfinished review hides every face the AI found.
- A face the AI never found has no box: a person has to draw it.
- The export doesn't run the AI again: it blurs the same boxes and regions you see in the preview, on every frame.

Export, step 1: a 41× faster blur

The export writes a new video, frame by frame, and at first it was slow: the blur alone took about four fifths of every frame. Now it shrinks the face region, blurs the small copy and enlarges it back.

One wide blur



61.9 ms

Shrink, blur, enlarge



1.5 ms

Per 576 × 324 region at 1080p. The result is a slightly different blur, at least as strong; solid black is the effect that removes everything.

Export, steps 2 and 3: stages, then the encoder

Reading, blurring and writing ran one after another for every frame. Now they are three stages that work at the same time, and FFmpeg does the writing.

One loop



52 frames/s

Three stages at once



74 frames/s

+ FFmpeg, Microsoft's H.264 encoder



147–167 frames/s

1,800 real frames of a 1080p59.94 camcorder file, a blur on every frame.

Why FFmpeg: the encoder and the sound

1. **OpenCV's writer**
   as the app used it, only OpenH264: about one CPU core, and no audio at all

2. **FFmpeg**
   streams the frames into Microsoft's H.264 encoder, a software encoder that spreads the work over several cores

3. **The sound**
   read straight from the original file, lined up with the first frame and ended with the last

4. **Why the ending matters**
   in a short test without it, FFmpeg kept going through the whole 23-minute soundtrack: 49 frames a second instead of 167

5. **4K60**
   a 4K60 crowd clip, 892 frames, exports in about 19 seconds

Today's limit: split work

GPU the AI (face detection)

CPU reading, blurring and writing the video

- A 4K frame is about 25 MB. It's converted from the video's colors on the way in and back again on the way out: at 4K, half of reading a frame is that conversion (6.7 of 13.0 ms).
- Reading the video on the GPU through OpenCV got slower in a quick test, probably because every frame had to come back to the CPU.
- So the next big step is to keep each frame on the GPU from start to finish: faster, but more complex, and a series of its own.

Reading one 4K frame: 13.0 ms



What's next for review

- A viewer's idea, not built yet: show on the timeline what the model scanned and what a person has reviewed, rescan just one stretch at every frame, and keep it unreviewed until a person confirms it.
- Today, the reviewer has to find those gaps.
- In short: the AI proposes, a person fills the gaps, and the export writes exactly what was reviewed.

Next episode

Episode 5: the engineering behind it, an architecture tour.

Where it's going: the whole pipeline on the GPU.

Footage: Towfiqu barbhuiya, William Fortunato, Talha Uguz / Pexels.

Samit Desai · linkedin.com/in/samit-desai-62868420